CONFIDENTIAL | INTERNAL DISCLOSURE | FOR INSTITUTIONAL REVIEW

PATENT APPLICATION DISCLOSURE DRAFT

Budgeted Contract Repair for Node-Based Agent Workflows

Constraint-Ranked Repair Candidates and Hash-Linked Execution Certificates

Drafting status (Revision 2). This document follows the fifteen-section disclosure format supplied for institutional review. It is a working draft and is neither legal advice nor a filing-ready specification. No assertion of novelty, inventive step or patentability is made: Section 5 records the prior-art position as open, and Section 12 states the proposed contribution expressly subject to a documented search and review by a patent professional. Applicant particulars are pre-filled from institute records and must be confirmed; inventor rows must be completed. Annexure A is internal review material that correlates each claimed element with the current implementation and must be removed before any filing.

1. Full name, nationality and address of applicant(s)

Full Name	Vishwakarma Institute of Technology
Nationality	Indian
Address	666, Upper Indiranagar, Bibwewadi, Pune, Maharashtra, India - 411 037

2. Full name (including middle name), nationality, address, mail id, phone number of inventor(s)

Full Name (including middle name)	Nationality	Address (department, institute)	Mail ID	Phone No.	Gender
Dr. Minal Barhate	Indian	Department of Artificial Intelligence and Data Science, Vishwakarma Institute of Technology, Pune			

The institute format requires a minimum of two faculty inventors in addition to the student inventors. Complete every row, state each inventor's contribution, and delete unused rows before filing.

3. Title of the invention

Budgeted Contract Repair for Node-Based Agent Workflows Using Constraint-Ranked Repair Candidates and Hash-Linked Execution Certificates.

4. Technical field of the invention

- Computer-implemented orchestration of software-agent and tool workflows represented as directed graphs.
- Runtime validation of typed data artifacts against machine-evaluable interface and policy contracts carried on graph edges.
- Selection and execution of workflow repairs under a remaining-execution-budget constraint, ranked jointly by cost, latency, privacy risk and residual failure risk.

- Integrity recording, by canonical hashing and Merkle aggregation, of automated repair decisions and their outcomes.
- Human-in-the-loop release gating and metered execution accounting for automated workflows.

5. Prior art

The following categories of prior art are material. Each statement identifies a capability of the category as commonly practised; a documented search has not yet been completed, so no reference is asserted as the closest art and no conclusion of novelty is drawn.

- **Workflow orchestration engines** - for example Apache Airflow, Temporal, Prefect and Dagster - schedule tasks expressed as directed graphs and provide retries, timeouts and static graph validation. Their recovery behaviour is typically triggered by task exceptions or exit codes, not by machine-evaluable contracts over the payload of a specific edge, and the recovery action is generally fixed by configuration rather than selected at run time from ranked alternatives under a remaining-budget constraint.
- **Data-quality and pipeline-contract frameworks** - for example Great Expectations, TensorFlow Extended example validation, and dbt data contracts - assert declared expectations about data at defined checkpoints. They report or block on a violation, but generally do not select among candidate repair operations whose costs are bounded by the remaining budget, and do not emit a hash-linked record binding the violation, the chosen remedy and the repaired output.
- **Agent orchestration libraries** - for example LangGraph, CrewAI and AutoGen - support graph-shaped agent execution, state passing and human-in-the-loop interrupts. They do not ordinarily attach to each edge a contract carrying freshness, privacy classification, provenance and cost ceilings that is evaluated against every artifact passing that edge.
- **Provenance and tamper-evident logging** - for example distributed tracing, hash-chained audit logs, and Merkle-tree transparency logs of the kind used in certificate transparency - provide integrity evidence for recorded events. Such records establish that events are unchanged; they do not by themselves bind a record to an automated decision that a failed contract was repaired and re-validated.
- **Automated program repair** - candidate patches are ranked by fitness against a test suite. The present mechanism differs in object (data artifacts at workflow edges rather than program source), in admissibility (candidates are bounded by remaining budget and by a privacy ceiling), and in verification (the repaired artifact is re-evaluated against the same contract that failed).

Gap addressed. The combination not found in the above categories is: diagnosis keyed to the individual violated contract clause, admissibility imposed as two hard filters before any ranking, ranking by a composite objective that weights privacy and residual risk above cost, mandatory re-validation against the original contract before downstream consumption, and a hash-linked certificate of the decision. Whether that combination is new or inventive is a question for the search and review recorded in Annexure B; it is not asserted here.

6. Objects of the invention

- To provide a workflow execution method in which every inter-node connection carries a machine-readable contract that is evaluated against each artifact passing over it.
- To produce a structured failure report that names the specific unsatisfied constraint rather than reporting a generic task failure.
- To admit a repair only when it addresses every reported violation code and its estimated cost fits within the remaining execution budget.
- To rank admissible repairs by a configurable objective over cost, latency, privacy risk and residual failure risk, so that the cheapest repair is not automatically chosen.
- To re-validate the repaired artifact against the original contract before any downstream node consumes it.

- To halt or escalate with a structured error instead of silently treating a failed contract as satisfied.
- To produce a hash-linked certificate binding the failure, the repair decision and the repaired artifact.
- To require an explicit human release decision on any path terminating at a final output node.
- To meter execution against a balance and refuse execution when the balance is insufficient.

7. Synopsis

The present invention relates to a computer-implemented method and system for executing a workflow of agent or tool nodes arranged as a directed graph, in which each edge between two nodes carries a machine-readable contract. The contract states what the consumer requires of the data arriving over that edge: the payload fields that must be present, whether provenance must accompany the data, a floor on a confidence value, a maximum privacy classification, a maximum age, and a ceiling on the cost that may have been incurred in producing the data. Each node emits an artifact - a payload together with confidence, privacy classification, provenance, creation time and incurred cost - and the artifact is evaluated against the contract of every outgoing edge before the receiving node is allowed to consume it.

When an artifact fails a contract, the system does not raise a generic error and does not retry blindly. It issues a failure report naming the violated constraints, then selects a remedy from a library of candidate repair operations. Two hard filters are applied before any preference is expressed: a candidate is admissible only if the set of violation codes it declares itself able to fix contains every code in the failure report, and only if its estimated cost does not exceed the remaining budget. Admissible candidates are ranked by a weighted objective over estimated cost, latency, privacy risk and residual failure risk, and the lowest-scoring candidate is applied. The repaired artifact is then re-evaluated against the same contract, and the receiving node proceeds only if that evaluation is clean; otherwise a bounded alternative is attempted or the execution terminates with a structured error. Finally, canonical hashes of the failure report, the decision, the repaired artifact and the execution trace are aggregated into a Merkle root and persisted, so that the repair decision can later be shown to be unaltered.

The system further provides a mandatory human release gate on paths that terminate at a final output node, and meters every execution and every repair against a running balance, refusing work that the balance cannot fund.

8. Brief description of drawings (if any)

Figure 1 is a block diagram of the proposed system, showing the workflow store, the pre-execution validator, the execution engine, the artifact record, the contract evaluator, the failure report, the repair planner, the repair applier, the re-validator and the certificate builder, together with the approval gate and the metered budget ledger.

Figure 2 is a flow diagram of candidate admission, ranking, repair and re-validation, including the rejection and termination branches.

Figure 3, in Section 15, shows the hash structure of the execution certificate.

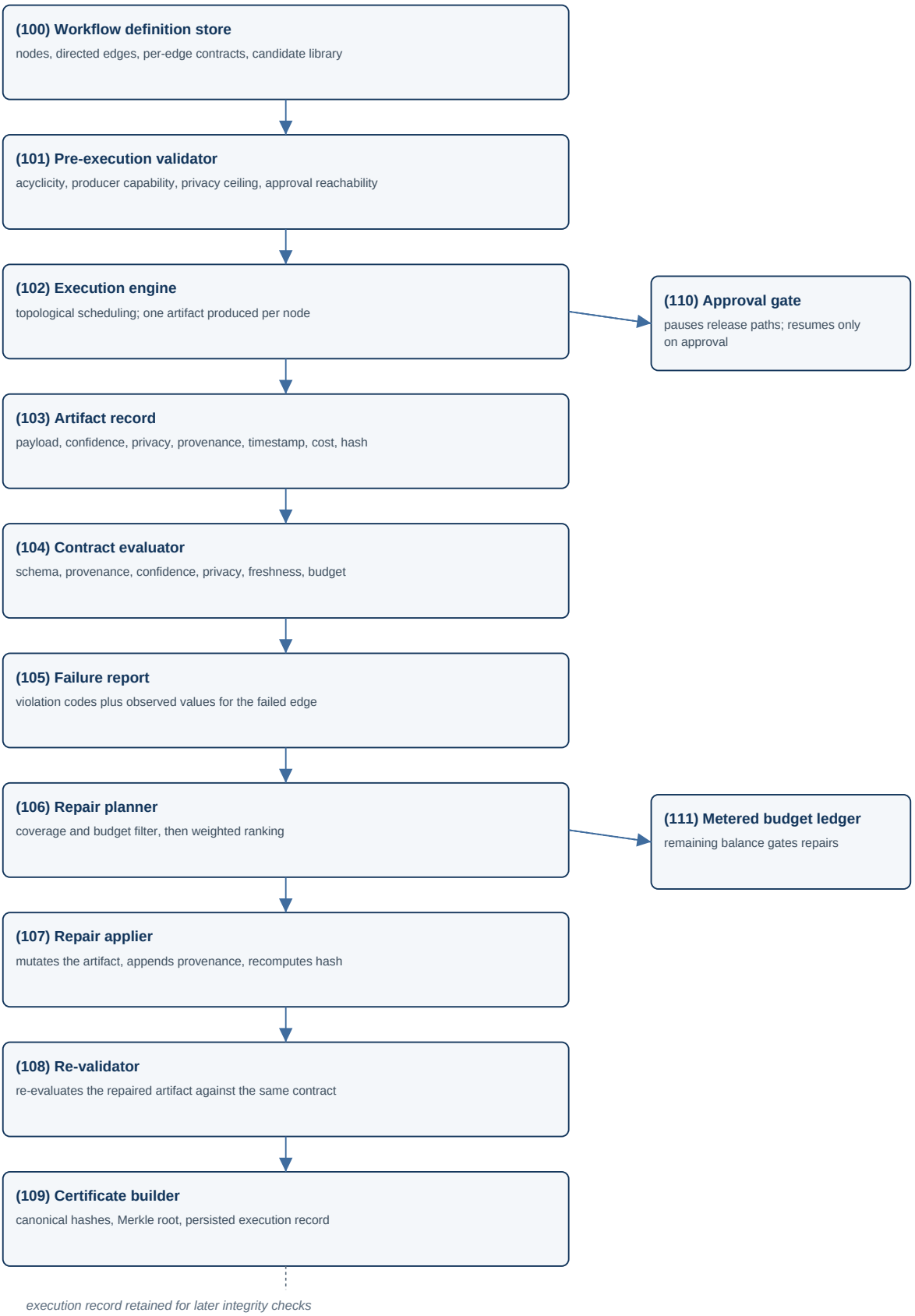


Figure 1. Block diagram of the proposed system. Reference numerals (100) to (109) denote the processing stages; (110) and (111) denote the release gate and the budget ledger, which act across stages.

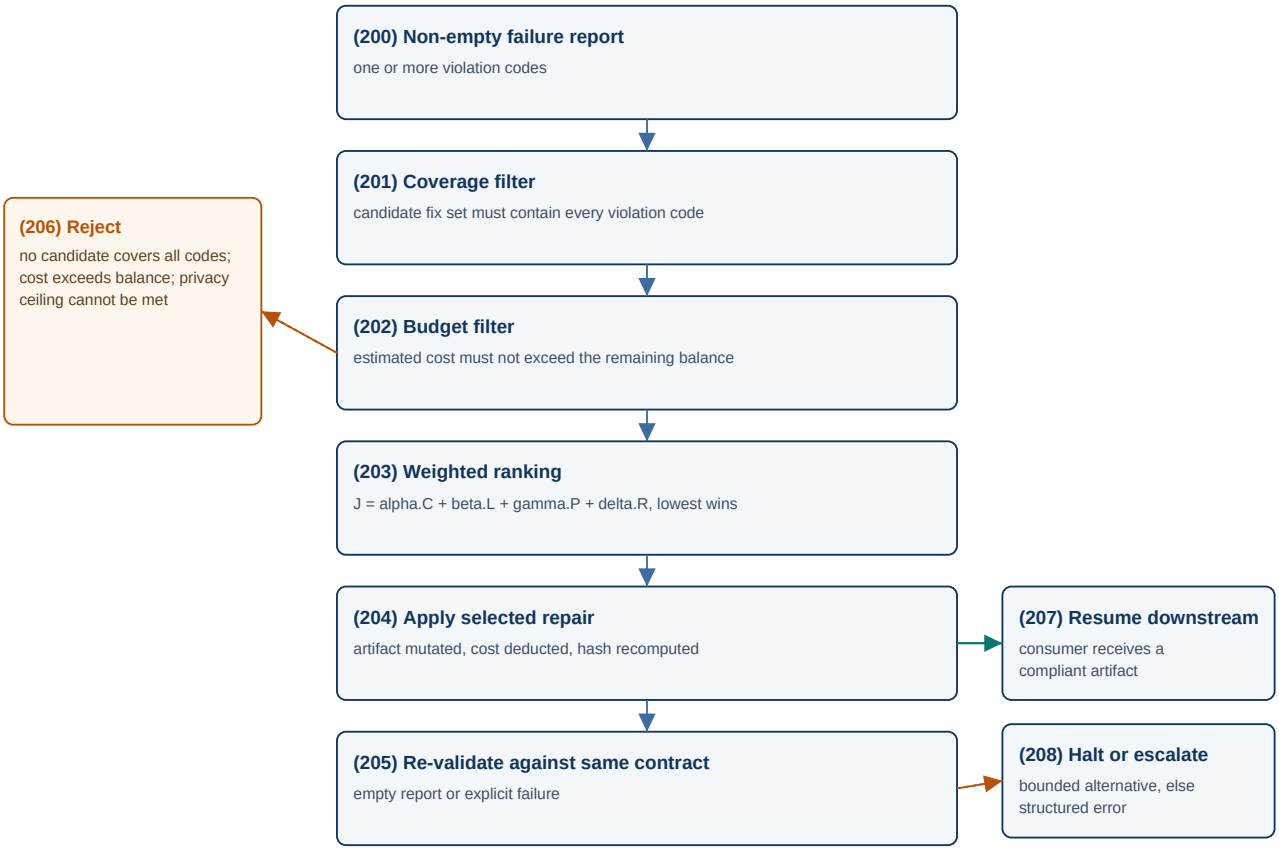


Figure 2. Candidate admission, ranking, repair and re-validation, with the rejection branch (206) and the two outcomes (207), (208).

9. Detail description of the invention

The present invention relates to a system and method for executing workflows of software agents and tools in which the connections between nodes are treated as enforceable contracts rather than as plain data channels, and in which a failure to satisfy such a contract is diagnosed, repaired under a budget, re-checked, and recorded in a tamper-evident form. The description below follows the reference numerals of Figure 1.

9.1 System components

A workflow definition store (100) holds a directed graph whose nodes each declare a node kind and whose edges each carry a contract. The same store holds a library of candidate repair operations, each declaring the violation codes it can address and an estimate of its cost, latency, privacy risk and residual failure risk. A pre-execution validator (101) examines the graph before any node runs. An execution engine (102) orders the nodes topologically and invokes one node at a time, producing an artifact record (103) for each. A contract evaluator (104) tests the artifact against the contract of each outgoing edge and emits a failure report (105) when a test fails. A repair planner (106) admits candidates and ranks them; a repair applier (107) mutates the artifact; a re-validator (108) repeats the evaluation. A certificate builder (109) hashes the relevant records. An approval gate (110) interrupts any path that leads to a final output node, and a metered budget ledger (111) tracks the remaining balance across both node execution and repair.

9.2 Contract model

A contract is a machine-readable record attached to a directed edge. It states what the consumer of that edge requires. The fields used in the present embodiment, and the violation code raised when each is not met, are set out below.

Field	Meaning	Violation code
required_fields	Payload keys that must be present in the artifact	schema
provenance_required	At least one provenance entry must be attached	provenance
min_confidence	Floor on the artifact confidence value	confidence
privacy	Maximum privacy classification permitted over the edge	privacy
freshness_seconds	Maximum permitted age of the artifact	freshness
max_cost	Ceiling on the cost incurred in producing the artifact	budget
fallback	Whether a repair may be attempted at all on this edge	not a code

The privacy field uses a total order: public is less restrictive than internal, which is less restrictive than confidential. A violation arises when the artifact classification exceeds the ceiling stated by the contract, or when the ceiling stated by the contract exceeds the maximum classification the receiving node declares it will accept.

9.3 Artifact model

An artifact is the unit of data that travels over an edge. It comprises a payload mapping field names to values; a provenance list recording the sources or prior artifacts from which it was derived; a confidence value between zero and one; a privacy classification drawn from the total order above; a creation timestamp; the cost incurred in producing it; and an integrity hash. The hash is computed over a canonical serialisation of every other field, so that any change to the payload or to the metadata changes the hash.

9.4 Pre-execution validation

Before execution the graph is checked in full, and every defect is reported rather than only the first. The checks are: that the graph contains no directed cycle; that every edge refers to nodes that exist; that every contract can be parsed; that each producing node declares the fields its outgoing contract requires, so that an edge cannot demand data the producer was never able to supply; that the privacy ceiling of each edge does not exceed the maximum classification declared by the receiving node; and that every node terminating the workflow is reachable from an approval node, so that no result can be released without a human decision. A graph failing any check is not executed.

In a further embodiment, the same checks are applied across nesting levels, so that a composite node referring to a stored sub-workflow is validated against the contracts at its boundary and a reference cycle between stored workflows is rejected rather than entered.

9.5 Execution and failure detection

Nodes are executed in topological order. When a node runs, it receives the artifacts produced by its predecessors, produces its own artifact, and incurs its declared cost against the ledger. Each outgoing edge of that node is then evaluated. The evaluator tests the payload for the required fields, the provenance list for the provenance condition, the confidence value against the floor, the classification against the ceiling, the age against the freshness window, and the incurred cost against both the contract ceiling and the remaining balance. The result is either a clean evaluation or a failure report containing one or more violation codes together with the observed values that produced them, so that the report explains itself without reference to the code that generated it.

9.6 Candidate admission and ranking

A repair is chosen in two stages. Admission is a hard filter with no trade-off: a candidate is considered only if the set of violation codes it declares it can fix contains every code in the failure report, and only if its estimated cost does not exceed the remaining balance. A candidate that fixes most but not all of the reported codes is not admitted, and neither is a candidate that can fix the failure but cannot be afforded. If no candidate is admitted, the system does not improvise: it halts or escalates according to the contract. Admission therefore separates correctness from preference, and no amount of preference can override it.

Among admitted candidates a scalar score is computed as a weighted sum of estimated cost C, estimated latency L, estimated privacy risk P and estimated residual failure risk R:

J = alpha.C + beta.L + gamma.P + delta.R

where alpha, beta, gamma and delta are non-negative and configurable. In the present embodiment the defaults are alpha = 1.0, beta = 0.2, gamma = 4 and delta = 6, so that a repair which worsens the privacy posture or which is likely to fail again is penalised more heavily than one which is merely expensive. The candidate with the lowest score is selected. Because admission has already removed the inadmissible candidates, the ranking expresses preference only within the admissible set, and ties are resolved by the fixed enumeration order of the library, which makes selection deterministic and reproducible.

9.7 Repair application and re-validation

Applying a repair mutates the artifact, not the program. In the present embodiment the operations available are: supplying deterministic placeholder values for absent required fields; raising the confidence value to at least the contractual floor plus a margin; appending a provenance entry that identifies the repair that was applied; setting the classification to the contractual ceiling; clamping the recorded cost; and refreshing the creation timestamp. After mutation the artifact hash is recomputed and the repair is charged against the balance.

The repaired artifact is then re-evaluated against the same contract that failed. This step is essential rather than optional: a repair is an estimate of a remedy, and the only evidence that the remedy worked is that the original test now passes. If the re-evaluation is clean, the receiving node consumes the repaired artifact and execution continues. If it is not clean, the system may attempt a bounded alternative - a small, fixed number of further admitted candidates - and otherwise terminates with a structured error naming the remaining violations. At no point is a failed contract recorded as satisfied.

9.8 Alternative embodiments

Repair by subgraph insertion. Instead of mutating the artifact, the selected repair may be a sub-workflow that is inserted between the producing and the receiving node. In that embodiment the system verifies the interface compatibility of the inserted sub-workflow, verifies that the augmented graph remains acyclic, executes the inserted sub-workflow within the remaining balance, and re-validates its output against the original contract. Artifact mutation and subgraph insertion share the same admission, ranking and re-validation discipline and differ only in where the repair is applied.

Repair by candidate composition. Where a failure report contains more than one violation code and no single candidate covers them all, the system may consider combinations of candidates whose union covers every code, scoring the combination by the same objective applied to the summed cost, the greatest latency and the product of the risk terms, and re-validating after the combination is applied.

Calibrated risk estimates. The four coefficients and the per-candidate risk estimates may be replaced by values calibrated from the recorded outcomes of earlier repairs on the same workflow, so that a candidate which has historically failed on this workflow is progressively penalised. The certificate records described below supply the history on which such calibration depends.

Escalation instead of relaxation. Where no admissible candidate satisfies the privacy ceiling, the system may route the artifact to a human review queue rather than relaxing the ceiling to make a repair admissible. Relaxation of a policy constraint is not treated as a repair.

Signed certificates. The certificate described in 9.10 may additionally be signed, so that a party holding the corresponding public key can verify not only that the record is unchanged but that it was produced by the system that claims to have produced it.

9.9 Human release gate and composite workflows

A node of the approval kind interrupts execution. The engine persists the execution in a waiting state together with the artifacts produced so far, and resumes only when an authenticated decision is received for that execution. Approval releases the result; rejection terminates the execution in a rejected state. Because pre-execution validation requires every terminal output node to be reachable from an approval node, a workflow that could release a result without a human decision is not executed at all.

A composite node encapsulates a previously stored workflow and may be used as a single node inside another workflow. It is executed recursively, with the artifacts arriving at the composite node supplying the seed inputs of the inner graph, and with approval nodes inside the inner graph suppressed, since the composite boundary is where the human decision belongs. The artifact leaving the composite node carries the payload and the provenance of the inner result, so that the inner derivation remains visible to the outer contract. A recursion-depth limit and a reference-cycle check prevent a stored workflow from invoking itself directly or through an intermediate workflow.

9.10 Certificate construction and the scope of its guarantee

When execution completes, the system constructs a certificate. Each of the following is reduced to a canonical serialisation - keys sorted, separators minimal, non-ASCII characters escaped, non-serialisable values stringified - and hashed: the original artifact, the contract together with the failure report, the admitted candidate set together with the selected candidate, the repaired artifact, and each event of the execution trace. These leaf hashes are combined pairwise by concatenation and re-hashing until a single root remains, an unpaired leaf at any level being duplicated. The root is stored with a format version, the trace hashes, the repair records, the final artifact hash, a certificate identifier, the execution identifier and a creation timestamp.

The guarantee this provides is precisely stated, because it is easy to overstate. Any later modification to a hashed record changes the root, and a party holding the record can recompute the root and detect the change. That is integrity. The certificate does not provide authenticity, since in the present embodiment it is not signed; it does not provide completeness, since omission of an event before hashing leaves no trace in the root; and it says nothing about whether the recorded payload is accurate or the repaired output is correct. A hash certifies that a record has not changed since it was hashed, and nothing more.

9.11 Metered budget

Every execution is charged against a balance held for the account. Before execution the nominal cost of the graph is compared with the balance and the run is refused if the balance is insufficient. During execution each node charges its declared cost, and each applied repair charges its estimated cost, so that the remaining balance used by the admission filter is always the balance that actually remains. A warning is presented as the balance approaches exhaustion, and execution is refused outright at zero. The balance is therefore not a display; it is an input to the admission decision, which is why the budget filter in 9.6 is a hard constraint.

9.12 Illustrative use

A workflow of topic input, researcher, evidence validator, draft writer, approval and final output is defined with contracts between consecutive nodes. The researcher produces an artifact whose confidence is 0.10 while the contract on its outgoing edge requires 0.80. The evaluator reports a single violation, code confidence. The library contains one candidate able to fix that code, a cross-checking operation estimated at cost 2, latency 2, privacy risk 0.02 and residual risk 0.04; it is admitted because the remaining balance exceeds 2, and its score under the default weights is 2 + 0.4 + 0.08 + 0.24 = 2.72. The artifact confidence is raised above the floor, its hash is recomputed and the repair is charged. Re-evaluation now returns a clean report, so the writer proceeds. The certificate binds the original artifact hash, the contract and failure digest, the selected candidate, the repaired artifact hash and the trace events under one root. Later, if any of those records is altered, recomputation of the root no longer matches the stored value.

Deterministic simulated agents may be substituted for live models for demonstration and for regression testing, in which case the artifacts are synthetic. Synthetic artifacts should be labelled as such wherever they are displayed, since provenance over synthetic data carries no evidential value.

10. Best method of performance of the invention

The invention is preferably performed as follows, in a single local or hosted service that holds the workflow definition, the candidate library and the balance.

1. Define the workflow as a directed graph, attaching to each edge a contract stating required payload fields, provenance requirement, confidence floor, privacy ceiling, freshness window, cost ceiling and whether repair is permitted.
2. Populate the candidate library, recording for each candidate the violation codes it can address and its estimated cost, latency, privacy risk and residual failure risk.
3. Validate the graph before execution and refuse the graph, listing every defect, if it contains a cycle, if an edge demands a field its producer cannot supply, if an edge exceeds the receiving node's privacy ceiling, or if a terminal output is not reachable from an approval node.
4. Compare the nominal cost of the graph with the balance and refuse the run if the balance is insufficient.
5. Execute the nodes in topological order, producing for each an artifact with payload, provenance, confidence, classification, timestamp, cost and hash, and charging the declared cost against the balance.
6. Evaluate each artifact against the contract of each outgoing edge and assemble a failure report containing every violation code and the observed values that produced it.
7. Where the report is clean, permit the receiving node to consume the artifact and continue.
8. Where the report is not clean, admit only those candidates whose fix set contains every reported code and whose estimated cost does not exceed the remaining balance.
9. Score the admitted candidates by J = alpha.C + beta.L + gamma.P + delta.R and apply the lowest-scoring candidate, charging its estimated cost against the balance.
10. Re-evaluate the repaired artifact against the same contract, and permit the receiving node to consume it only if that evaluation is clean; otherwise attempt a bounded alternative and then terminate with a structured error.
11. Where the path terminates at a final output node, pause and persist the execution in a waiting state, and release the result only on an authenticated approval decision.
12. Hash the original artifact, the contract and failure report, the candidate decision, the repaired artifact and the trace events, aggregate them into a Merkle root, and persist the certificate with its version, identifiers and timestamp.

11. Claims

The claims below are drafted for review. Claim 1 is the independent method claim; claims 2 to 11 depend from it; claims 12 and 13 are the system and medium claims. Claim elements that the current prototype does not yet perform are identified in Annexure A and must be implemented or removed before filing.

1. A computer-implemented method for executing a node-based workflow, comprising: (a) storing a directed workflow graph comprising a plurality of nodes and directed edges, each edge being associated with a machine-readable contract specifying at least a set of required payload fields and a cost ceiling; (b) validating the graph before execution to confirm that the graph is acyclic, that each contract is parseable, that each producing node declares the fields required by its outgoing contract, and that each terminal output node is reachable from a node of an approval kind; (c) executing the nodes in topological order, each node producing an artifact comprising a payload, a confidence value, a privacy classification, a provenance list, a creation timestamp and an incurred cost; (d) evaluating an artifact produced at a producing node against the contract of an outgoing edge to produce a structured failure report comprising one or more violation codes selected from the set consisting of schema, provenance, confidence, privacy, freshness and budget; (e) responsive to a non-empty failure report, admitting a candidate only from a library of repair candidates each declaring a set of violation codes it addresses and an estimated cost, an estimated latency, an estimated privacy risk and an estimated residual failure risk, wherein admission requires that the declared set of violation codes contains every violation code of the failure report and that the estimated cost does not exceed a remaining execution budget; (f) computing, for each admitted candidate, a score as a weighted sum of its estimated cost, estimated latency, estimated privacy risk and estimated residual failure risk using non-negative weights, and selecting the admitted candidate having the lowest score; (g) applying the selected candidate to the artifact to obtain a repaired artifact and deducting the estimated cost of the selected candidate from the remaining execution budget; (h) re-evaluating the repaired artifact against the same contract and permitting a consuming node of the edge to consume the repaired artifact only when the re-evaluation produces an empty failure report, and otherwise attempting a bounded number of further admitted candidates and thereafter terminating execution with a structured error; and (i) recording a certificate comprising a cryptographic hash of the failure report, of the selected candidate, of the repaired artifact and of each event of an execution trace, and a root value derived from those hashes.

2. The method of claim 1, wherein the contract further specifies at least one of a provenance requirement, a minimum confidence value, a maximum privacy classification, a maximum artifact age and a flag indicating whether repair is permitted on the edge.

3. The method of claim 1, wherein the privacy classification is ordered from public to internal to confidential, and a privacy violation code is raised when the classification of the artifact exceeds the maximum stated by the contract or when the maximum stated by the contract exceeds the maximum classification declared by the consuming node.

4. The method of claim 1, wherein a freshness violation code is raised when the difference between a current time and the creation timestamp of the artifact exceeds a maximum age stated by the contract.

5. The method of claim 1, wherein a budget violation code is raised when the incurred cost of the artifact exceeds the cost ceiling stated by the contract or exceeds the remaining execution budget.

6. The method of claim 1, wherein, when no candidate is admitted and the contract does not permit repair, execution is terminated and the failure report is returned together with the evaluated candidate set, without recording the contract as satisfied.

7. The method of claim 1, wherein applying the selected candidate comprises supplying deterministic values for absent required fields, raising the confidence value to at least the minimum stated by the contract, appending to the provenance list an entry identifying the selected candidate, and recomputing the hash of the artifact.

8. The method of claim 1, wherein the root value is a Merkle root obtained by pairwise hashing of leaf hashes, an unpaired leaf at any level being duplicated.

9. The method of claim 1, wherein a node of the approval kind causes execution to be suspended with a persisted waiting state and to be resumed only upon receipt of an authenticated approval decision for that execution, a rejection decision terminating the execution in a rejected state.

10. The method of claim 1, wherein, when the failure report comprises a plurality of violation codes and no single admitted candidate addresses all of them, admission further comprises evaluating combinations of candidates whose combined declared violation codes contain every code of the failure report, and scoring each combination by the said weighted sum applied to the combined estimated cost, the greatest estimated latency and the combined estimated risks.

11. The method of claim 1, wherein a composite node encapsulates a stored workflow graph and is executed recursively with the artifacts arriving at the composite node supplying seed inputs, nodes of the approval kind being suppressed within the stored graph, and execution being bounded by a recursion depth limit.

12. A system for executing node-based workflows, comprising one or more processors and memory storing instructions that, when executed, cause the system to perform the method of any one of claims 1 to 11.

13. A non-transitory computer-readable medium storing instructions that, when executed by one or more processors, cause performance of the method of any one of claims 1 to 11.

12. Inventive step of your invention

The proposed technical contribution, stated as a combination rather than as any single feature, is the following.

- Diagnosis at the granularity of an individual edge contract, so that the failure report names the specific unsatisfied constraint rather than reporting that a task failed.
- Admissibility imposed as two hard filters applied before any preference is expressed: full coverage of the reported violation codes, and cost within the remaining budget. A candidate that partly fixes the failure, or that cannot be afforded, is excluded rather than ranked low.
- Ranking within the admissible set by a composite objective that weights privacy risk and residual failure risk above cost, so that the cheapest remedy is not automatically chosen.
- Mandatory re-evaluation of the repaired artifact against the original contract before any downstream consumption, with explicit halt or escalation where the remedy does not work, in place of treating a repair as having succeeded because it was applied.
- A hash-linked certificate binding the failure, the admission decision, the selected candidate and the repaired artifact under one root, so that the decision can later be shown to be unaltered.
- Combination of the above with a mandatory human release gate on terminal output paths and with a metered budget that participates in admission rather than merely being displayed.

Status of this statement. The foregoing identifies what the applicants presently regard as the technical contribution. It is not a conclusion that the contribution is new or involves an inventive step. Those questions depend on a documented prior-art search, on a comparison of the closest references claim by claim, and on review by a patent professional; none of the three has yet been completed, and Annexure B records them as outstanding.

13. Industrial application

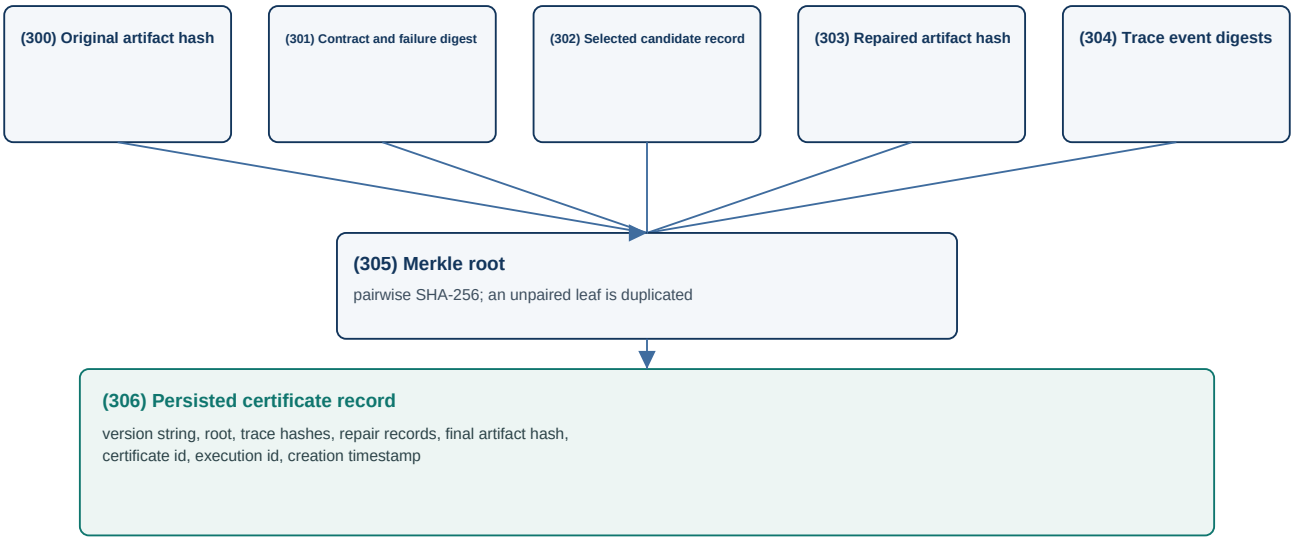
- **Agent and tool orchestration platforms** - for running multi-step automated work where a downstream step depends on the quality, freshness and confidentiality of what arrives from an upstream step.
- **Research and evidence-processing pipelines** - where every produced statement must retain provenance and where a low-confidence intermediate result must not pass downstream unchecked.
- **Document review and content operations** - where privacy classification and retention limits apply to intermediate material.

- **Regulated and audited workflows** - where a substituted or repaired intermediate result must be demonstrably recorded, and where the record must later withstand an integrity check.
- **Metered software services** - where each execution and each recovery step is charged to an account and where work must stop when funds are exhausted.
- **Data engineering and analytics** - where pipeline contracts between producing and consuming stages are declared and where a violated contract must be remedied within a cost limit rather than by rerunning an entire pipeline.

14. Abstract

A computer-implemented method and system execute a workflow of agent or tool nodes arranged as a directed graph in which each edge carries a machine-readable contract stating required payload fields, provenance, a confidence floor, a privacy ceiling, a freshness window and a cost ceiling. Each node emits an artifact carrying a payload, confidence, classification, provenance, timestamp and incurred cost, and every artifact is evaluated against the contract of each outgoing edge. On a failure, a report naming the violated constraints is produced, and a repair is admitted from a candidate library only when it addresses every reported violation and its estimated cost fits the remaining budget. Admitted candidates are ranked by a weighted sum of cost, latency, privacy risk and residual failure risk, and the lowest-scoring one is applied. The repaired artifact is re-evaluated against the same contract before a downstream node may consume it. Canonical hashes of the failure, the decision, the repaired artifact and the trace are aggregated into a Merkle root and persisted as a certificate. A human release gate and a metered budget bound execution.

15. Drawing



Hashes establish that a record is unchanged. They do not establish that it is true or complete.

Figure 3. Hash structure of the execution certificate.

Figure 3 illustrates the certificate constructed at stage (109) of Figure 1. The original artifact produced by the failing node contributes a hash (300). The contract together with the structured failure report contributes a digest (301). The admitted candidate set together with the selected candidate contributes a digest (302). The repaired artifact contributes its hash (303). Each event of the execution trace contributes a digest (304). These leaf hashes are combined pairwise and re-hashed, an unpaired leaf at any level being duplicated, until a Merkle root (305) remains. The root is stored in a persisted certificate record (306) together with a format version string, the trace hashes, the repair records, the final artifact hash, a certificate identifier, the execution identifier and a creation timestamp. Because every leaf participates in the root, an alteration to any recorded artifact, decision or trace event causes a recomputation of the root to disagree with the stored value, which is the integrity property the certificate provides.

Annexure A. Implementation status of the claimed elements (internal review)

This annexure is not part of the disclosure and must be deleted before any filing. It exists so that the applicants can see which claimed elements the present prototype actually performs. A claim element marked "not implemented" cannot be relied upon as disclosed subject matter until the corresponding behaviour exists and is tested; a claim element marked "partial" is performed only in some cases.

Claim element	Current implementation	Status
1(a) contracts with required fields and cost ceiling	Contract model and contract lookup on each edge	Implemented
1(b) pre-execution validation	Graph validation: cycles, dangling edges, producer capability, privacy ceiling, approval reachability	Implemented
1(c) artifact with confidence, privacy, provenance, timestamp, cost	Artifact model and artifact constructor	Implemented
1(d) six-code failure report	Contract evaluator returning violation codes with observed values	Implemented
1(e) coverage and budget admission	Candidate generator and selector with fixed-coverage rule	Partial - a single candidate must cover every code, so a multi-code failure admits nothing
1(f) weighted ranking	Selector with weights 1.0, 0.2, 4.0, 6.0	Implemented
1(g) apply repair and charge	Repair operation mutating the artifact and recomputing its hash	Implemented as artifact mutation only
1(h) re-validate repaired artifact	No call of the evaluator after the repair is applied	NOT IMPLEMENTED
1(i) certificate with root	Certificate builder and Merkle aggregation	Partial - created only when at least one repair occurred and the run is not paused for approval
2-5 contract field semantics	Evaluator checks per field	Implemented
6 structured halt when nothing is admissible	Structured error returned with the candidate set	Implemented
7 repair operation detail	Repair operation	Implemented
8 Merkle root	Merkle aggregation	Implemented
9 approval pause and resume	Execution pause with persisted waiting state, and an authenticated decision endpoint	Implemented
10 candidate composition	No combination search	NOT IMPLEMENTED
11 composite node with recursion limit	Recursive execution of a stored graph	Partial - no recursion depth limit and no reference-cycle check, so a self-referencing composite node would recurse without bound
Metered budget	Balance checks and charging on execution and repair	Partial - the charge is recomputed by scanning trace costs, and a clamp can conceal an overrun

Annexure B. Pre-filing checklist

Item	Owner	Status
Applicant particulars confirmed	Applicant	Pre-filled - confirm
Inventor list complete, including at least two faculty	Applicants	Open
Documented prior-art search performed and recorded (databases, queries, date of search)	Inventors	Open
Closest references compared against each claim, differences stated	Inventors and reviewer	Open
Element 1(h), re-validation, implemented and tested	Inventors	Open
Element 10, candidate composition, implemented or claim removed	Inventors	Open
Element 11, recursion limit and reference-cycle check, implemented	Inventors	Open
Certificate produced for every completed execution	Inventors	Open
Claim set and description reviewed by a patent professional	Institution	Open
Drawing rules checked for the intended filing jurisdiction	Institution	Open
Annexure A deleted	Inventors	Open

Nothing in this document asserts that the invention is patentable, novel or inventive, and nothing in it should be treated as legal advice. The statements of technical fact were correlated with the prototype identified in Annexure A at the time of drafting; that correlation must be repeated after any change to the implementation.